Microsoftのネイティブアプリ回帰戦略に関する調査報告書：アーキテクチャの転換、組織的矛盾、および成功の実現可能性

1. 序論：Satya Nadellaの「ファン奪還」宣言とネイティブ回帰への文脈

2026年4月29日に行われたMicrosoftの2026会計年度第3四半期（Q3）決算発表において、最高経営責任者（CEO）のSatya Nadellaは、コンシューマー向けWindowsビジネスを戦略の中心に据えるという明確な方針を打ち出した[1]。同CEOは、月間アクティブなWindowsデバイスがWindows 10やそれ以前のバージョンを含めて16億台を突破した事実に触れつつ、「我々のコンシューマービジネスに関して、Windows、Xbox、Bing、Edge全体でファンを取り戻し（win back fans）、エンゲージメントを強化するために必要な基礎的作業（foundational work）を行っている」と明言した[2]。この「ファンを取り戻す」という発言は、単なるマーケティングメッセージではなく、昨今のWindowsエコシステムに対するユーザーの不満が、企業のリスクマネジメントにおける重大な課題にまで発展していることを示す異例の公式な認容である[4]。

長年、Windowsは企業や一般消費者にとって「避けられないインフラ」であり、ユーザーはオペレーティングシステムに多少の不具合や使いにくさがあっても、それを許容してきた。しかし、Windows 11のリリース以降、この暗黙の妥協関係は崩れつつある。Windows 11は厳格なハードウェア要件とクリーンな視覚言語をもたらしたが、同時に、OS自体がMicrosoftのクラウドサービスやサブスクリプションを押し付けるための巨大な広告塔のように振る舞うようになったという批判に直面した[4]。特に、スタートメニューの柔軟性の欠如、タスクバーの機能制限、コンテキストメニューの遅延など、インターフェースの一貫性のなさと応答性の悪さがユーザーの「筋肉の記憶（muscle memory）」に反する形となって現れた[4]。

このような状況を打破するための最優先の技術的対策として浮上しているのが、「ネイティブアプリへの回帰」である。Microsoftのパートナーアーキテクトであり、Microsoft Storeやファイルエクスプローラーの開発に携わるRudy Huynは、Windowsアプリを開発する新たなチームの結成を発表し、それがProgressive Web Apps（PWA）ではなく「100%ネイティブ」な体験を提供すると確約した[5]。また、.NETやASP.NET Coreの設計で知られる著名なエンジニアであるDavid Fowlerも、ソーシャルメディア上で「Native apps are BACK!（ネイティブアプリが帰ってきた！）」と宣言し、開発者コミュニティに大きな波紋を呼んだ[2]。

しかし、「Microsoftがネイティブアプリに早急に回帰することは難しい」という仮説は、テクノロジーの歴史、Microsoft内部の組織構造、およびソフトウェア開発の現在の経済学を鑑みると、極めて妥当であると言える。本報告書では、Microsoftが推し進めるネイティブ回帰戦略の全貌、それを支える技術的基盤（WinUI 3、.NET 10 Native AOT）、社内に存在するWebラッパー戦略（WebView2）との致命的な矛盾、そしてこの戦略が最終的に成功するかどうかの実現可能性について、多角的な視

点から徹底的な分析を行う。

2. アーキテクチャ上の妥協：Webラッパー技術の台頭とその代償

Microsoftがネイティブアプリへの回帰を急務としている背景には、過去数年間にわたってソフトウェア業界全体を席巻してきた「Webベースのラッパー技術」への過度な依存がある。Electron、WebView2、React Nativeといったフレームワークは、開発者が単一のWebコードベースを使用してWindows、macOS、Linux向けのデスクトップアプリケーションを迅速に展開することを可能にした。このクロスプラットフォーム開発のアプローチは、企業のエンジニアリングリソースを大幅に節約し、機能の展開を加速させたが、その代償はすべてエンドユーザーのパフォーマンスとシステムリソースに転嫁された [4]。

ユーザーに課せられた「パフォーマンスの税金」

Webラッパー技術を使用して構築されたアプリケーションは、本質的にバックグラウンドで独立したブラウザ（Chromiumベースのエンジンなど）のインスタンスを起動している。これにより、アプリケーションがアイドル状態であっても、深刻なシステムリソースの浪費が発生している。

アプリケーション / 機能	アーキテクチャ技術	リソース消費とパフォーマンスへの影響	ユーザー体験における主要な摩擦
WhatsApp (デスクトップ版)	WebView2 (PWA)	アイドル時でも最大600MBのRAMを消費（8GB RAM環境時）[5]	バックグラウンドでの常駐によるシステム全体のメモリ圧迫
Discord	Electron	アクティブ時に最大4GBのRAMを消費 [5]	メモリリークによるクラッシュ、重いゲームのバックグラウンド実行時のパフォーマンス低下
Microsoft Copilot (Windows)	WebView2	バックグラウンドで500MB、使用時は最大1GBのRAM消費 [5]	OSの組み込み機能であるにもかかわらず、スタンドアロンのブラウザと同等の負荷
Microsoft Store	混成（Webコンポー	ページ遷移ごとに数	ネイティブなUWPア

	ネント含む）	秒の遅延が発生[7]	プリからの退行と認識される顕著なロード遅延

このようなアーキテクチャの蔓延により、Windowsは次第に「ブラウザがオペレーティングシステムのふりをしている」かのように機能するようになった[4]。ユーザーは、ファイルエクスプローラーを開いたり、システムの設定パネルを操作したりする日常的なタスクにおいて、あたかも小さなビデオゲームを起動しているかのようなレイテンシ（遅延）とメモリ消費を強いられている[4]。2026年Q3の決算発表でSatya Nadellaが「低メモリデバイス向けのパフォーマンス改善」を特に強調したのは[2]、RAM容量が4GBから8GBの一般的なオフィス用PCや教育機関向けデバイスにおいて、このWebアーキテクチャによるリソースの肥大化が限界に達していることを示している。

さらに、プログレッシブWebアプリ（PWA）は開発が軽量である反面、ネイティブアプリに備わっている堅牢なオフラインモードや、OSのシステムレベルの通知、ファイルシステムへの深い統合といった重要な機能が欠如していることが多い[5]。Windows 11のUIそのものにもReactベースのコンポーネントが入り込んでおり、これが結果としてスタートメニューの表示遅延といった、ユーザーのプラットフォームに対する信頼感を根底から揺るがす事態を引き起こしている[6]。

3. ネイティブ回帰を牽引する技術基盤：.NET 10とWinUI 3の成熟

「100%ネイティブ」への回帰というスローガンを現実のものとするため、Microsoftは開発プラットフォームとランタイム環境の根本的な再構築を進めている。この取り組みの技術的な中核となるのが、Windows App SDK（WinUI 3）の進化と、2025年11月にリリースされた最新のランタイムフレームワークである「.NET 10」の導入である[8]。

迷走の歴史を越えたGUIフレームワーク：WinUI 3

Windowsのグラフィカルユーザーインターフェース（GUI）開発の歴史は、戦略の転換とフレームワークの放棄が繰り返された「迷走の歴史」であると、多くの専門家や元Microsoftのエンジニアが指摘している[9]。Win32（MFC）の堅牢さから始まり、WPF（Windows Presentation Foundation）によるXAMLの導入、そしてWindows Phoneの失敗とともに崩壊したUWP（Universal Windows Platform）へと至る過程で、開発者は常に「次にどのフレームワークに投資すべきか」という不安に晒されてきた。

フレームワーク	登場時期と主な特徴	現在の位置づけと課題
Win32 / MFC	1990年代〜。C++による低レベルアクセスと最高のパフォーマンス。	非常に堅牢だが開発の生産性が低く、モダンなUIの構築が極めて困難[10]。

WPF	2006年～。XAMLによる高度なデータバインディングとUI設計。	企業向けアプリで広く使われているが、実行時のコード生成への依存が強く、最新のAOTコンパイルとの互換性に限界がある[11]。
UWP	2015年～。PC、モバイル、Xbox共通のプラットフォーム。	Windows Phoneの終焉によりエコシステムが崩壊。Microsoft自身の主力アプリも採用を見送った[9]。
WinUI 3 (Windows App SDK)	2021年～。Win32の柔軟性とモダンなFluent Designの統合。	現在のMicrosoftの主力ネイティブフレームワーク。段階的に機能パリティを達成し、最新のOSコンポーネントに採用されている[12]。

現在、Microsoftがネイティブアプリの未来を託しているのがWinUI 3である。2026年4月にリリースされたWindows App SDK 2.0では、XAML条件付きコンパイルや最新のストレージAPIが導入され[14]、さらにWinUI 3 Gallery 2.9のリリースにより、MicaやAcrylicといったFluent Designのマテリアルを任意のXAML要素に配置できるSystemBackdropElementがサポートされた[15]。Rudy Huynのチームは、このWinUI 3を活用し、まずはWindows 11のスタートメニューやタスクバー、ファイルエクスプローラーといったOSの「シェル（外殻）」にあたる部分からReactベースのコードを排除し、完全なネイティブコードへの書き換えを進行させている[4]。

###.NET 10とNative AOTによるパフォーマンスの飛躍

フレームワークの刷新に加え、ネイティブ回帰戦略の真の技術的ブレイクスルーをもたらしているのが、.NET 10における「Native AOT（Ahead-of-Time compilation：事前コンパイル）」の成熟である[4]。

従来の.NETアプリケーションはJIT（Just-In-Time）コンパイラを採用しており、アプリケーションの起動時に中間言語（IL）を実行環境に合わせて機械語に変換していた。このアプローチはプラットフォーム間の柔軟性を提供する一方で、起動時の遅延（ウォームアップ時間）とJITコンパイラ自体をメモリにロードするための高いメモリオーバーヘッドという代償を伴っていた[17]。対照的に、Native AOTはビルド時に直接プラットフォーム固有のネイティブな機械語バイナリを生成する。

.NET 10のNative AOT実装は、以下のような顕著な利点を提供する。

1. **即時的な起動速度:** JITコンパイルが不要になるため、アプリケーションの起動がほぼ瞬時に行われる。これはOSのコア機能や頻繁に呼び出されるランチャーにおいて決定的な優位性となる[17]。
2. **メモリフットプリントの大幅な削減:** 小規模なアプリケーションでは、バイナリサイズが数MBに収まり、実行時のメモリ消費も劇的に減少する。これにより、低リソースのデバイスでも軽量な動作が可能となる[18]。
3. **自立型実行と依存関係の排除:**.NETランタイムフレームワークがシステムにインストールされて

いない環境でも動作するため、配布と管理が極めてシンプルになる[19]。

さらに、.NET 10のランタイム自体も、JITのインライン化、メソッドの非仮想化、スタック割り当ての拡張、AVX10.2ハードウェアアクセラレーションのサポートなど、コード生成と最適化の面で大規模な改善が施されている[20]。2026年5月のWindows 11アップデート（KB5083631）において、ファイルエクスプローラーのメモリリークが修正され、31年前のレガシーなプロパティダイアログがWinUI 3ベースのコンポーネントに置き換えられたことは[5]、これらの技術スタックが机上の空論ではなく、実際のOSの安定性向上に寄与し始めていることを証明している。

4. 組織内の深刻な戦略的矛盾：Microsoft 365部門におけるWeb技術の強行

OS開発を担うWindows部門がWinUI 3と.NET 10を用いて「100%ネイティブ」への回帰を強力に推し進めている一方で、Microsoft社内にはそれに真っ向から対立する巨大な潮流が存在する。それが、Officeエコシステムやコラボレーションツールを統括するMicrosoft 365部門による「Web技術への全面的な依存戦略」である。この組織間の決定的な矛盾こそが、ユーザーが指摘する「Microsoftがネイティブアプリに早急に回帰することは難しい」という見解を完全に裏付ける最大の証拠である。

「One Outlook」（Project Monarch）の現実とエンタープライズの反発

Microsoftは現在、旧来のWin32アーキテクチャに基づく「クラシックOutlook」を段階的に廃止し、新しいOutlook（内部コードネーム：Project Monarch）への完全移行を計画している[22]。この「One Outlook」戦略の目的は、Windows、macOS、Webブラウザのすべての環境で単一のコードベースを使用し、機能の一貫性を保つことにある。しかし、この新しいOutlookはネイティブアプリではなく、実態はOutlook Web App（OWA）をベースにし、デスクトップ上でWebView2を介してレンダリングされるラッパーアプリである[23]。

Microsoft 365部門が新しいOutlookにおいてネイティブ開発（WinUI 3など）ではなくWebView2を採用した理由は、経済性と機能展開の迅速性にある。単一のWebコードベースを使用することで、開発リソースを劇的に削減できるだけでなく、クラウドベースの最新機能（特にMicrosoft 365 CopilotなどのAI機能）を、クライアント側のソフトウェアアップデートを待たずにサーバーサイドの変更だけで即座に全ユーザーに展開することが可能となる[25]。

しかし、このアーキテクチャの転換は、特にエンタープライズ市場において深刻な反発を招いている。最大の障壁は、新しいOutlookが「COMアドイン」を一切サポートしていないことである[26]。多くの企業は、顧客関係管理（CRM）システム、コンプライアンス維持のための電子情報アーカイブ、セキュリティ監視ツールなど、日常の業務ワークフローをWin32版クラシックOutlookのCOMアドインに深く依存している。これらのアドインはWebベースの新しいアドインアーキテクチャ（Office JavaScript API）に簡単に移植できるものではなく、企業のIT部門は移行を強く躊躇している。

このエンタープライズの抵抗を受け、Microsoftは移行スケジュールの見直しを余儀なくされた。2026年2月末、Microsoftは管理センター（MC949965）を通じて、エンタープライズ環境における新しいOutlookの「オプトアウト（デフォルト化され、手動で戻す設定）」フェーズの開始時期を、当初予定していた2026年4月から**2027**年**3**月へと約**1**年間延期すると発表した[26]。さらに、クラシックOutlook自

体のサポートは、サブスクリプションおよび永続ライセンスの双方で少なくとも2029年まで継続されることが確約されている[26]。これは、Microsoftの主力アプリケーションにおいて、ネイティブからWeb技術への移行が極めて難航しており、完全な移行には数年単位の時間が必要であることを如実に示している。

Microsoft TeamsにおけるWebアーキテクチャの永続化

同様の戦略的アプローチは、Microsoftのもう一つの主力生産性アプリケーションであるMicrosoft Teamsにも見られる。近年リリースされた新しいTeams（Teams 2.0 / 3.0）は、以前のバージョンで批判の的となっていた重厚なElectronフレームワークから脱却し、起動速度とメモリ効率を向上させた。しかし、これはネイティブアプリ（WinUIやC++）への移行を意味するものではなく、単に基盤を**Microsoft Edge WebView2**に切り替えたに過ぎない[28]。

2026年に順次導入されているTeamsの新機能群——例えば、メールアドレスのみを利用した外部ユーザーとのシームレスなチャット機能や、複数のテナント・組織からの通知を単一のフィードに統合するマルチテナント（MTMA）アーキテクチャ、さらにはOmnissa（旧VMware）などの仮想デスクトップインフラ（VDI）向け最適化——は、すべてこのWebView2によるクラウド接続アーキテクチャを前提として構築されている[29]。

これらの事実は、Microsoft内部における深刻な分断を示している。Windows部門がいくらOSのレイテンシを削り、スタートメニューや設定画面をWinUI 3でネイティブ化して「基礎的な作業」を完了させたとしても、ユーザーが日常業務の大半を費やすOutlookやTeamsが巨大なWebView2プロセスとして動作し続ける限り、システム全体のパフォーマンスとリソース消費を劇的に改善することは不可能である。OSのコア層はネイティブに回帰しても、アプリケーション層は依然としてWeb技術に強く縛られているのが現状である。

5. AI PC時代とCorePCアーキテクチャが求める「OSの透明化」

Satya Nadellaが「ファンを取り戻す」ためのネイティブ回帰を支持し、Windowsのパフォーマンス改善を約束した背景には、単に従来のPCユーザーを満足させるという以上の、より巨大で長期的なパラダイムシフトのビジョンが存在する。それは「AI PC（Copilot+ PC）」の普及と、それに伴う「Agentic AI（自律型AIエージェント）」の台頭を見据えたOSの再定義である。

OSリソースをNPUとAIエージェントに明け渡すための軽量化

Microsoftは次世代のWindows（広くWindows 12または内部コードネーム「CorePC」として知られる）に向けて、OSのアーキテクチャをコンポーネント化（モジュール化）し、より軽量で安全な基盤を構築する計画を進めている[32]。このモジュール化されたOSでは、読み取り専用のシステムパーティションや隔離されたドライバー領域が導入され、アップデートの迅速化とセキュリティの強化が図られる[33]。

このアーキテクチャ変革の重要な駆動力となっているのが、Copilot+ PCの要件である強力なNPU（Neural Processing Unit）の存在である。Microsoftは、OSが単なるクラウドサービスのシンクライアントとして機能するのではなく、「エッジ（ローカル端末）における無制限のインテリジェンスの基盤」と

なる未来を構想している[34]。例えば、「Recall」のような機能は、バックグラウンドで常にユーザーの画面活動を解析し、セキュアなVBS（Virtualization-based Security）エンクレーブとTPMによって暗号化されたローカルのベクトルデータベースを構築する[35]。

このような常時稼働する重いAI処理がバックグラウンドで行われる環境下では、OSの基本的なシェル（スタートメニュー、タスクバー、通知センター）や設定アプリが、WebView2やElectronを使って無駄なメモリ（RAM）やCPU、そしてNPUリソースを消費することは許されない。AIワークロードにPCのコンピューティングリソースを最大限に割り当てるため、OSのインターフェース要素は極限まで軽量化されたネイティブコード（.NET 10 Native AOTとWinUI 3）で記述され、ユーザーの作業の「邪魔にならない（getting out of the way）」透明なインフラストラクチャにならなければならないのである[4]。

SaaSの崩壊とAIネイティブ・ビジネスアプリケーションの未来

さらに、Satya Nadellaは最近のポッドキャストのインタビューで、「AIエージェントの時代において、従来のSaaS（Software as a Service）やビジネスアプリケーションの概念そのものが崩壊する可能性がある」という極めて大胆な予測を行っている[36]。

Nadellaによれば、従来のビジネスアプリケーションの根幹は、単なるCRUD（Create: 作成、Read: 読み取り、Update: 更新、Delete: 削除）データベースのフロントエンドに、独自のビジネスロジックをハードコードしたものである。しかし、今後のエージェント型AIの進化により、これらのビジネスロジックは「AI層（AI tier）」へと完全に移行することになる[37]。ユーザーは、経費精算アプリや人事管理アプリといった個別のアプリケーション画面を立ち上げて手動でデータを入力するのではなく、Microsoft Agent 365などによって統制されたAIエージェントに対して自然言語やコンテキストを通じて指示を出し、エージェントがバックエンドの複数のデータベースやAPIを横断して自律的に処理を実行するようになる[36]。

この未来図においては、肥大化したWebベースのグラフィカルユーザーインターフェース（GUI）を持つ伝統的なアプリケーションは徐々に中抜きされ、その存在意義を失っていく。OSが提供すべき究極の価値は、重いアプリケーションランチャーとしての機能ではなく、背後でAIエージェントが極めて高速かつ安全に動作するための「極めて軽量でネイティブな基盤」を提供することに他ならない。これこそが、Nadellaが「基礎的な作業（foundational work）」と呼ぶ戦略的シフトの真意であり、OSシェル部分のネイティブ化を急ぐ最大の理由である。

6. サードパーティエコシステムへの波及と開発者の経済学

Microsoft自身のファーストパーティアプリやOSコアがネイティブに回帰する動きを見せている一方で、この戦略がサードパーティの開発者エコシステム全体に波及するかどうかは、別の複雑な問題をはらんでいる。

クロスプラットフォームの経済的優位性

現代のソフトウェア開発の経済学において、独立系ソフトウェアベンダー（ISV）やスタートアップ企業がWindows専用の「100%ネイティブ」アプリケーションをゼロから構築するインセンティブは極めて乏しい。Discord、Slack、Notion、Spotifyといった世界的な人気を誇るデスクトップアプリケーションのほぼすべてが、ElectronやReact NativeなどのWebベース技術を使用している。理由は単純であり、1つのWebコードベースを維持するだけで、Windows、macOS、Linux、さらにはモバイルやWeb

ブラウザ向けのアプリケーションを同時に、かつ同一の開発チームでデプロイできるからである[39]。

さらに、Microsoft自身も、社内の別の開発部門を通じて「React Native for Windows」というオープンソースプロジェクトを強力に推進し続けている[40]。2026年にはバージョン0.81および0.82がリリースされ、Metaの新しいFabricアーキテクチャへの移行や、ARM64デバイス上のAI画像分類器のサンプルアプリ構築など、Web技術を用いたWindowsデスクトップアプリ開発のサポートを強化している[41]。これは、Microsoftがサードパーティに対しては依然として「Web技術ベースのクロスプラットフォーム開発」を現実的な解として提供し続けていることを示しており、一貫して「100%ネイティブ」を強要しているわけではない。

開発者コミュニティの根強い不信感と参入障壁

また、前述した「GUIフレームワークの迷走の歴史」により、開発者コミュニティの間に深く根付いたMicrosoftに対する不信感も、ネイティブ回帰の大きな足かせとなっている。WPFからUWPへ、そしてWinUI 3へと推奨されるフレームワークが次々と変更され、そのたびに過去の投資が無に帰してきた経験から、多くの開発者は「今のMicrosoftの新しいネイティブフレームワークに投資しても、また数年後には別のアーキテクチャに置き換えられてしまうのではないか」という疑念を抱いている[39]。

サードパーティの開発者が、使い慣れたWebラッパー技術を捨ててネイティブなWindows開発（WinUI 3やNative AOT）に回帰するためには、単なる「Windowsファンのため」という情緒的な理由ではなく、圧倒的な経済的合理性が必要となる[4]。具体的には、WinUI 3による開発ツール（Visual Studio）の圧倒的な生産性の向上、Microsoft Storeを通じたユーザー獲得におけるアルゴリズム上の明確な優遇措置、あるいは低メモリPCやARMベースのデバイス（Snapdragon X搭載のCopilot+ PCなど）における圧倒的なパフォーマンス差による新規ユーザー層の獲得など、プロダクトマネージャーを説得できるだけの「測定可能な優位性」が提示されなければならない[4]。現状では、この経済的な計算式を覆すだけのインセンティブは十分に提供されていない。

7. 結論：ユーザーの仮説の検証と**Microsoft**の戦略的着地点

本報告書における技術的、組織的、および経済的な要因の多角的な分析結果に基づくと、「**Microsoft**がネイティブアプリに早急に回帰することは難しい」というユーザーの事前の見立ては、極めて正確であり完全に支持される。Microsoftの「ネイティブアプリへの回帰」というスローガンは、1990年代のWin32時代のような「すべてのアプリケーションがWindows専用のネイティブコードで記述される」というノスタルジックなソフトウェアのルネサンスを意味するものでは決してない。

Microsoftの真の戦略的着地点は、全面的なネイティブ回帰ではなく、「特定の領域における**100%**ネイティブ化」と「他の領域における**Web**技術の意図的な永続化」という、極めて高度なハイブリッド構造の完成である。

この結論は、以下の3つのレイヤーにおける異なるアプローチによって裏付けられる。

1. **OS**シェルおよび組み込みインフラ層（成功が確実視される領域）：
   Rudy Huynのチームが焦点を当てているWindows 11のスタートメニュー、タスクバー、ファイルエクスプローラー、設定パネル、および極小規模なユーティリティアプリについては、「100%ネイティブ」への回帰が急速かつ確実に進行している。WinUI 3の機能拡充と.NET 10 Native AOTの技術的成熟は、この領域におけるレイテンシ問題やメモリ肥大化を解決するのに十分

な強力な武器となっている。2026年から2027年にかけて、ユーザーはOSの基本操作における顕著なレスポンス向上と「軽さ」を体感することになるだろう。これはSatya Nadellaの言う「ファンを取り戻す」ための最も重要な第一歩として成功を収めるはずである。

2. 第一者エンタープライズ・生産性アプリ層（ネイティブ回帰が行われない領域）：
ユーザーが日常の作業時間の大部分を費やすOutlookやMicrosoft Teams、そして中心的なインターフェースとなるMicrosoft Copilotについては、ネイティブアプリ（WinUI 3によるフルスクラッチでの書き直し）への回帰は発生しないと断言できる。新しいOutlookのエンタープライズ移行が2027年3月に延期されたことは、Microsoftのビジネス戦略がすでに「WebView2を基盤とした単一コード・マルチプラットフォーム展開」の上に成り立っており、この巨大な技術的・経済的投資を覆してネイティブに戻ることは不可能であることを示している。彼らはOSシェル層をネイティブ化して削り出したシステムリソース（RAMとCPU/NPU）を、これら巨大なWebベースの生産性アプリを安定稼働させるための余裕として活用する戦略を採っている。
3. サードパーティ・アプリケーション層（現状維持の領域）：
クロスプラットフォーム開発の経済的優位性が崩れない限り、サードパーティ開発者がWindows専用のネイティブアプリ開発に大規模に回帰することは考えにくい。開発者は引き続きElectronやReact NativeなどのWebラッパー技術を利用し続けるだろう。

総括すると、Satya NadellaとMicrosoftのエンジニアリングチームが目指しているのは、「OSそのものがリソースを浪費するのをやめ、インフラストラクチャとして透明な裏方に徹する」ことである。OSのコアインターフェースを極限まで軽量なネイティブコードで再構築することでプラットフォームへの信頼を回復し、そこで確保されたハードウェアの余力を、クラウドと結びついた巨大なWebView2アプリや、将来的にビジネスソフトウェアのあり方そのものを変革する「自律型AIエージェント」に提供する。これこそが、Microsoftが推進する「ネイティブアプリ回帰」の真の姿であり、この限定的かつ戦略的な範囲においてのみ、彼らの取り組みは成功を収めることになるだろう。

引用文献

1. Microsoft Fiscal Year 2026 Third Quarter Earnings Conference Call, 5月 8, 2026にアクセス、https://www.microsoft.com/en-us/investor/events/fy-2026/earnings-fy-2026-q3
2. Microsoft admits Windows 11 lost its way, Nadella pledges to "win back fans" | TechSpot, 5月 8, 2026にアクセス、https://www.techspot.com/news/112258-microsoft-admits-windows-11-lost-way-nadella-pledges.html
3. Microsoft CEO confirms "foundational work" to "win back" fans - OC3D, 5月 8, 2026にアクセス、https://overclock3d.net/news/software/microsoft-ceo-confirms-foundational-work-to-win-back-fans-of-windows-and-xbox/
4. Microsoft Windows 11 Native Apps Comeback: Nadella's 'Win Back Fans' Plan, 5月 8, 2026にアクセス、https://windowsforum.com/threads/microsoft-windows-11-native-apps-comeback-nadellas-win-back-fans-plan.416180/
5. Microsoft engineer says native apps are back, and it could finally revive Windows 11's fight against web apps, 5月 8, 2026にアクセス、

https://www.windowslatest.com/2026/04/29/microsoft-engineer-says-native-apps-are-back-and-it-could-finally-revive-windows-11s-fight-against-web-apps/
6. Microsoft plans to build 100% native apps for Windows 11, as web apps ruin the OS experience - Windows Latest, 5月 8, 2026にアクセス、https://www.windowslatest.com/2026/03/28/microsoft-plans-to-build-100-native-apps-for-windows-11-as-web-apps-ruin-the-os-experience/
7. Why is Microsoft switching to Webview even on system apps (built-in apps)? : r/Windows11, 5月 8, 2026にアクセス、https://www.reddit.com/r/Windows11/comments/1pkuoeg/why_is_microsoft_switching_to_webview_even_on/
8. Announcing .NET 10 - Microsoft Developer Blogs, 5月 8, 2026にアクセス、https://devblogs.microsoft.com/dotnet/announcing-dotnet-10/
9. Microsoft Hasn't Had a Coherent GUI Strategy Since Petzold | Jeffrey Snover's blog, 5月 8, 2026にアクセス、https://www.jsnover.com/blog/2026/03/13/microsoft-hasnt-had-a-coherent-gui-strategy-since-petzold/
10. WinUI vs WPF, WinForms, and UWP | ComponentOne - mescius, 5月 8, 2026にアクセス、https://developer.mescius.com/blogs/winui-vs-wpf-winforms-uwp-and-mfc
11. WinUI vs WPF in 2026: A Practical Comparison for .NET Desktop Developers | CTCO, 5月 8, 2026にアクセス、https://www.ctco.blog/posts/winui-vs-wpf-2026-practical-comparison/
12. Windows developer FAQ - Windows apps | Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/windows/apps/get-started/windows-developer-faq
13. WinUI is an evolution of UWP which is an evolution of WinRT. WinUI has been arou... | Hacker News, 5月 8, 2026にアクセス、https://news.ycombinator.com/item?id=44766374
14. Windows App SDK 2.0 release notes - Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/windows/apps/windows-app-sdk/release-notes/windows-app-sdk-2-0
15. WinUI 3 Gallery 2.9 Highlights Windows App SDK 2.0 for App Developers, 5月 8, 2026にアクセス、https://visualstudiomagazine.com/articles/2026/05/01/winui-3-gallery-2-9-highlights-windows-app-sdk-2-0-for-app-developers.aspx
16. .NET 10 and the Next Leap in Native AOT for Cloud-Native Apps - DEV Community, 5月 8, 2026にアクセス、https://dev.to/arthus15/net-10-and-the-next-leap-in-native-aot-for-cloud-native-apps-a0k
17. Native AOT in .NET 10: Everything for C# Developers - DEV Community, 5月 8, 2026にアクセス、https://dev.to/chandana_pushpakumara_4bf/native-aot-in-net-10-everything-for-c-developers-2m7e
18. blown away by .NET10 NativeAOT : r/dotnet - Reddit, 5月 8, 2026にアクセス、

https://www.reddit.com/r/dotnet/comments/1r9uguu/blown_away_by_net10_nativeaot/
19. Native AOT deployment overview - .NET | Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/
20. What's new in .NET 10 runtime - Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/runtime
21. What's new in .NET 10 - Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
22. Too Many Microsoft Outlooks? Clear Up the Confusion in Simple Terms - Office Watch, 5月 8, 2026にアクセス、https://office-watch.com/2025/microsoft-outlook-versions-explained/
23. Overview of the new Outlook for Windows | Microsoft Learn, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/microsoft-365-apps/outlook/overview-new-outlook-windows
24. Microsoft to Replace Outlook for Mac With New Universal Client - MacRumors, 5月 8, 2026にアクセス、https://www.macrumors.com/2021/01/05/outlook-for-mac-to-be-replaced-with-new-client/
25. Outlook for Windows WebView2 Shift Triggers Windows 11 Backlash and Native Alternatives, 5月 8, 2026にアクセス、https://windowsforum.com/threads/outlook-for-windows-webview2-shift-triggers-windows-11-backlash-and-native-alternatives.396419/
26. 5 Outlook Problems and Issues IT Teams Are Dealing With in 2026 – TrustedTech, 5月 8, 2026にアクセス、https://www.trustedtechteam.com/blogs/software/outlook-problems-issues
27. Microsoft Delays By a Year Its New Outlook Rollout, 5月 8, 2026にアクセス、https://www.directionsonmicrosoft.com/microsoft-delays-by-a-year-its-new-outlook-rollout/
28. The New Microsoft Teams Client, 5月 8, 2026にアクセス、https://learn.microsoft.com/en-us/microsoftteams/platform/resources/teams-updates
29. Microsoft Teams in 2026: What's New, What's Changing, and What Businesses Must Prepare For - Reliable IT Support & VOIP Solutions in Maryland, DC and Florida - Teltek Systems, 5月 8, 2026にアクセス、https://myteltek.com/blog/2025/12/microsoft-teams-in-2026-whats-new-whats-changing-and-what-businesses-must-prepare-for/
30. What's New in Microsoft Teams | March 2026 | Microsoft Community Hub, 5月 8, 2026にアクセス、https://techcommunity.microsoft.com/blog/microsoftteamsblog/what%E2%80%99s-new-in-microsoft-teams--march-2026/4506985
31. What's New in Microsoft Teams | April 2026, 5月 8, 2026にアクセス、https://techcommunity.microsoft.com/blog/microsoftteamsblog/whats-new-in-microsoft-teams--april-2026/4515907
32. What's Coming in Windows 12? 6 Expert Predictions You Can Bet On | PCMag, 5月

8, 2026にアクセス、
https://www.pcmag.com/news/whats-coming-in-windows-12-expert-predictions-you-can-bet-on
33. Windows 12 AI First: Copilot Everywhere, NPUs, and Modular CorePC, 5月 8, 2026にアクセス、
https://windowsforum.com/threads/windows-12-ai-first-copilot-everywhere-npus-and-modular-corepc.402773/
34. Microsoft admits that Windows 11 has lost its way - iGuRu.gr, 5月 8, 2026にアクセス、
https://en.iguru.gr/microsoft-paradechetai-oti-windows-11-echasan-ton-dromo-tous/
35. Copilot+ PCs and AI Features for Businesses - Microsoft, 5月 8, 2026にアクセス、
https://www.microsoft.com/en-us/windows/business/devices/copilot-plus-pcs
36. Microsoft CEO: AI Agents Will Transform SaaS as We Know It - CX Today, 5月 8, 2026にアクセス、
https://www.cxtoday.com/customer-analytics-intelligence/microsoft-ceo-ai-agents-will-transform-saas-as-we-know-it/
37. Beyond SaaS: How AI-Native Business Apps Will Reshape Enterprise Technology, 5月 8, 2026にアクセス、
https://studiospace.com/blog/beyond-saas-how-ai-native-business-apps-will-reshape-enterprise-technology_173
38. Microsoft Build 2025: CEO Nadella Takes Platform, Systems Approach To The 'Agentic Web', 5月 8, 2026にアクセス、
https://www.crn.com/news/ai/2025/microsoft-build-2025-ceo-nadella-takes-platform-systems-approach-to-the-agentic-web
39. Developer explains why Windows 11 keeps getting web apps instead of native apps, 5月 8, 2026にアクセス、
https://www.windowslatest.com/2026/04/07/developer-explains-why-windows-11-keeps-getting-web-apps-instead-of-native-apps/
40. React Native for Desktop - Windows - Microsoft Learn, 5月 8, 2026にアクセス、
https://learn.microsoft.com/en-us/windows/dev-environment/javascript/react-native-for-windows
41. React Native Windows v0.82 is here!!, 5月 8, 2026にアクセス、
https://devblogs.microsoft.com/react-native/%F0%9F%9A%80react-native-windows-v0-82-is-here/
42. React Native Windows v0.81 is here!!, 5月 8, 2026にアクセス、
https://devblogs.microsoft.com/react-native/%F0%9F%9A%80react-native-windows-v0-81-is-here/